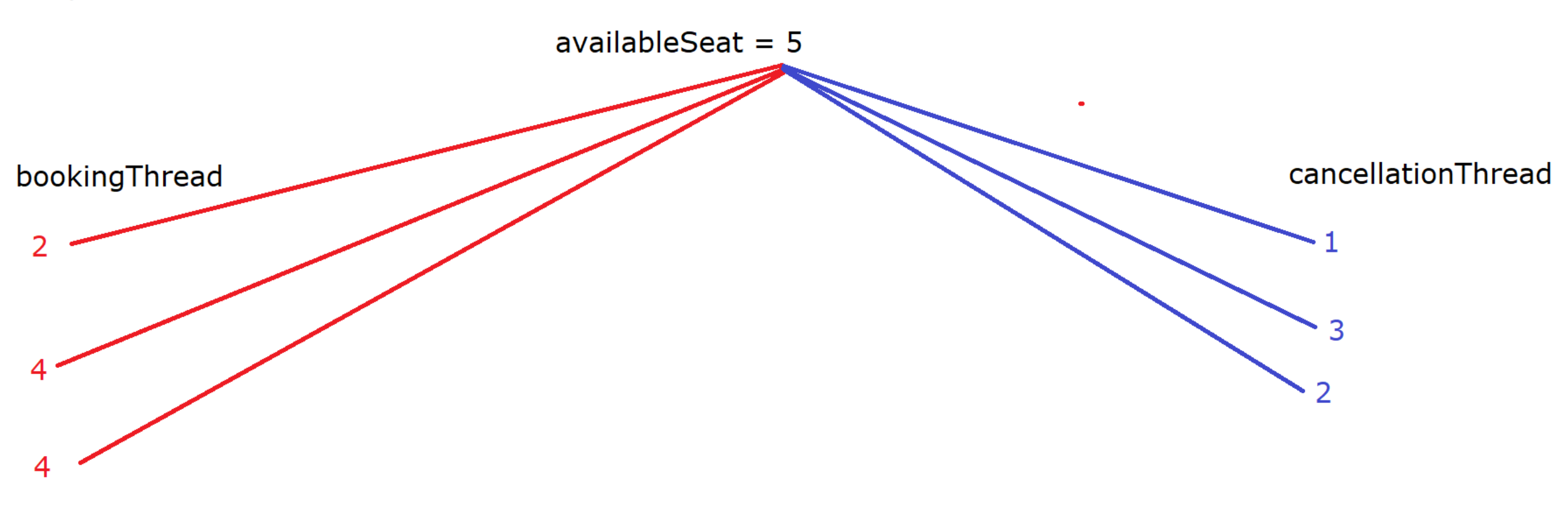
WAP to develop one Ticket Booking app where multiple threads can book the ticket as well as on the other hand multiple threads can cancel the ticket.



```java
package com.ravi.itc;

class TicketSystem
{
    private int availableTicket = 5;  //availableTicket = 5

    public synchronized void bookTicket(int noOfTickets) //noOfTickets = 4
    {
        while(noOfTickets > availableTicket)
        {
            System.err.println("Tickets are not available, Waiting for cancellation");
            try
            {
                wait();
            }
            catch(InterruptedException e)
            {
                e.printStackTrace();
            }
        }

        this.availableTicket -= noOfTickets;
        IO.println(noOfTickets+" tickets are booked, Available tickets are :"+availableTicket);
    }

    public synchronized void cancelTicket(int noOfTickets) //noOfTickets = 2
    {
        this.availableTicket += noOfTickets;
        IO.println(noOfTickets+" tickets are canecelled, Available tickets are :
"+this.availableTicket);
        notify();
    }


}

public class ITCDemo5
{
    public static void main(String[] args)
    {
        TicketSystem ticketSystem = new TicketSystem();  //lock is created

        Thread bookingThread = new Thread()
        {
            @Override
            public void run()
            {
                int []tickets = {2, 4, 4};

                for(int ticket : tickets)  //3rd Iteration [ticket = 4]
                {
                    ticketSystem.bookTicket(ticket);

                    try
                    {
                        Thread.sleep(1000);
                    }
                    catch(InterruptedException e)
                    {
                        e.printStackTrace();
                    }
                }

            }
        };

        bookingThread.start();


        Thread cancellationThread = new Thread()
        {
           @Override
           public void run()
           {
               int []tickets = {1,3,2};  // 3rd Iteration [ticket = 2]

               for(int ticket : tickets)
               {
                   ticketSystem.cancelTicket(ticket);

                   try
                   {
                       Thread.sleep(1200);
                   }
                   catch(InterruptedException e)
                   {
                       e.printStackTrace();
                   }

               }
           }
        };

        cancellationThread.start();










    }
 }
```

Program on notifyAll() methods :

---

```java
package com.ravi.itc;

class Resource
{
    private boolean flag = false;

    public synchronized void waitMethod()
    {
        IO.println("Wait");

        while (!flag) //Infinite Loop -> Finite Loop
        {
          try
          {
             IO.println(Thread.currentThread().getName() + " is waiting...");
             IO.println(Thread.currentThread().getName()+" is Waiting for Notification");
             wait();  //Child1  Child2  Child3
          }
          catch (InterruptedException e)
          {
                e.printStackTrace();
          }
        }//End of while loop

        IO.println(Thread.currentThread().getName() + " thread completed!!");
    }


    public synchronized void setMethod()
    {
        IO.println("notifyAll");
        this.flag = true;
        IO.println(Thread.currentThread().getName() + " has make flag value as a true");
        notifyAll();
    }
}

public class ITCDemo6
{
    public static void main(String[] args)
        {

        Resource r1 = new Resource(); //lock is created

        Thread t1 = new Thread(() -> r1.waitMethod(), "Child1");
        Thread t2 = new Thread(() -> r1.waitMethod(), "Child2");
        Thread t3 = new Thread(() -> r1.waitMethod(), "Child3");

        t1.start();
        t2.start();
        t3.start();

        try
        {
            Thread.sleep(2000);
        }
        catch (InterruptedException e)
        {
            e.printStackTrace();
        }

        //Another Thread
        Thread setter = new Thread(() -> r1.setMethod(), "Setter_Thread");
        setter.start();
    }
}
```

ThreadGroup :

------------------

It is a predefined class available in java.lang Package from JDK 1.0V.

By using ThreadGroup class we can put 'n' number of threads into a single group to perform some common/different operation.

By using ThreadGroup class constructor, we can assign the name of group under which all the thread will be executed.

ThreadGroup tg = new ThreadGroup(String groupName);

ThreadGroup class has provided the following methods :

public String getName() : To get the name of the Group

pubic int activeCount() : How many threads are alive and running under that particular group.

Thread class has provided constructor to put the thread into particular group.

Thread t1 = new Thread(ThreadGroup tg, Runnable target, String name);

By using ThreadGroup class, multiple threads will be executed under single group.

Thread class has provided a Method :

public ThreadGroup getThreadGroup() :

--

* It is a predefined method of thread class which return thread group Object.

//Programs :

--------------

```java
package com.ravi.thread_group;

public class ThreadGroupDemo1 {

    public static void main(String[] args)
    {
        Thread t = Thread.currentThread();

        IO.println("Current Thread name is :"+t.getName()); //main
        IO.println("Current thread group name is :"+t.getThreadGroup().getName()); //main

    }

}
```

```java
package com.ravi.thread_group;

class Batch60 implements Runnable
{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();

        IO.println(name+" thread is running under Batch 60 group");
        try
        {
            Thread.sleep(100);
        }
        catch(InterruptedException e)
        {
            e.printStackTrace();
        }
    }
}

public class ThreadGroupDemo2 {

    public static void main(String[] args)
    {
        ThreadGroup group = new ThreadGroup("Batch_60");
        Batch60 batch = new Batch60();

        Thread t1 = new Thread(group, batch,"Scott");
        Thread t2 = new Thread(group, batch,"Smith");
        Thread t3 = new Thread(group, batch,"Alen");
        Thread t4 = new Thread(group, batch,"John");

        t1.start();  t2.start();  t3.start();   t4.start();

        IO.println("Thread group name is :"+group.getName());
        IO.println("Active threads under this group :"+group.activeCount());


    }

}
```

`WAP to assign different threads which are runing into a particular group.`

```java
package com.ravi.thread_group;

class Tatkal implements Runnable
{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();
        IO.println(name+ " has booked the ticket under Tatkal Scheme");
    }

}

class PremimumTatkal implements Runnable
{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();
        IO.println(name+ " has booked the ticket under Premimum Tatkal Scheme");
    }
}

public class ThreadGroupDemo3
{
    public static void main(String[] args)
    {
        ThreadGroup tatkal = new ThreadGroup("Tatkal");
        ThreadGroup PremimumTatkal = new ThreadGroup("Premimum Tatkal");

        Tatkal obj = new Tatkal();
        PremimumTatkal obj1 = new PremimumTatkal();

        Thread scott = new Thread(tatkal, obj, "Scott");
        Thread alen = new Thread(PremimumTatkal, obj1, "Alen");

        scott.start();
        alen.start();

    }

}
```

```java
package com.ravi.thread_group;

public class ThreadGroupDemo4
{
    public static void main(String[] args)
    {
        Thread t = Thread.currentThread();
        IO.println(t.toString());

        t.setName("Parant_Thread");
        t.setPriority(Thread.MAX_PRIORITY);
        IO.println(t.toString());

    }

}
```